

OFS Fusion Splicer Machine Learning

AI in Fusion Splicing

Presented by: Andy Li

> TOC

Introduction to CNN

Past Model

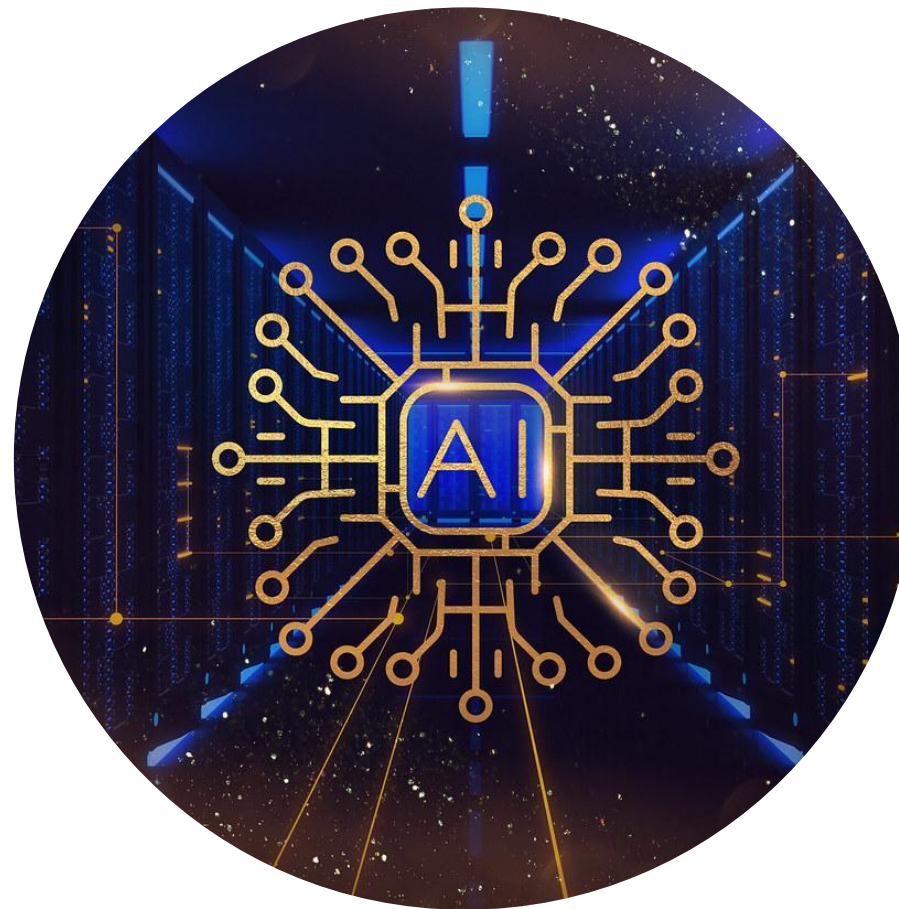
3 Fiber Model and function

Adding Additional Fibers

Other Models and Comparison

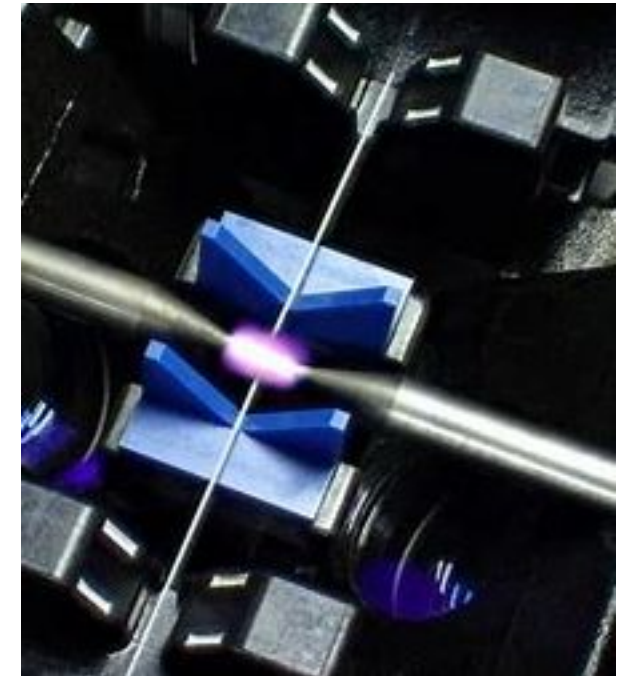
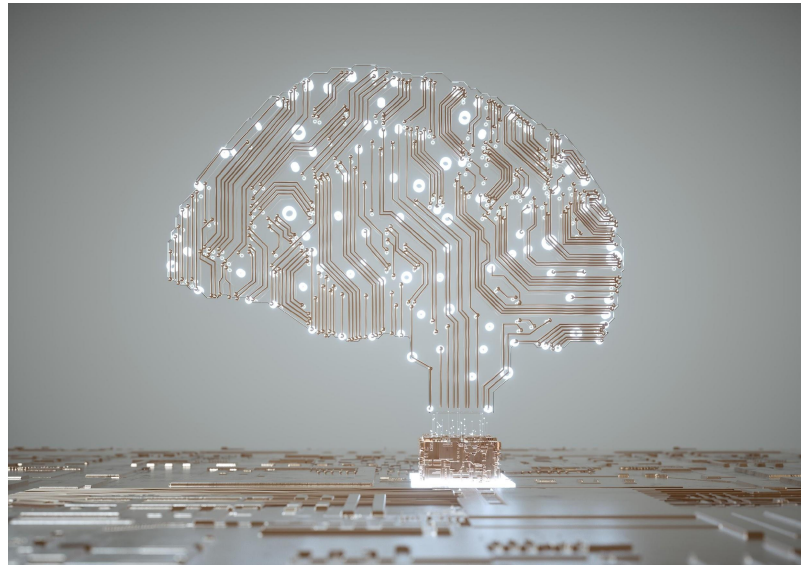
4 Fiber Model

Future Models and Applications



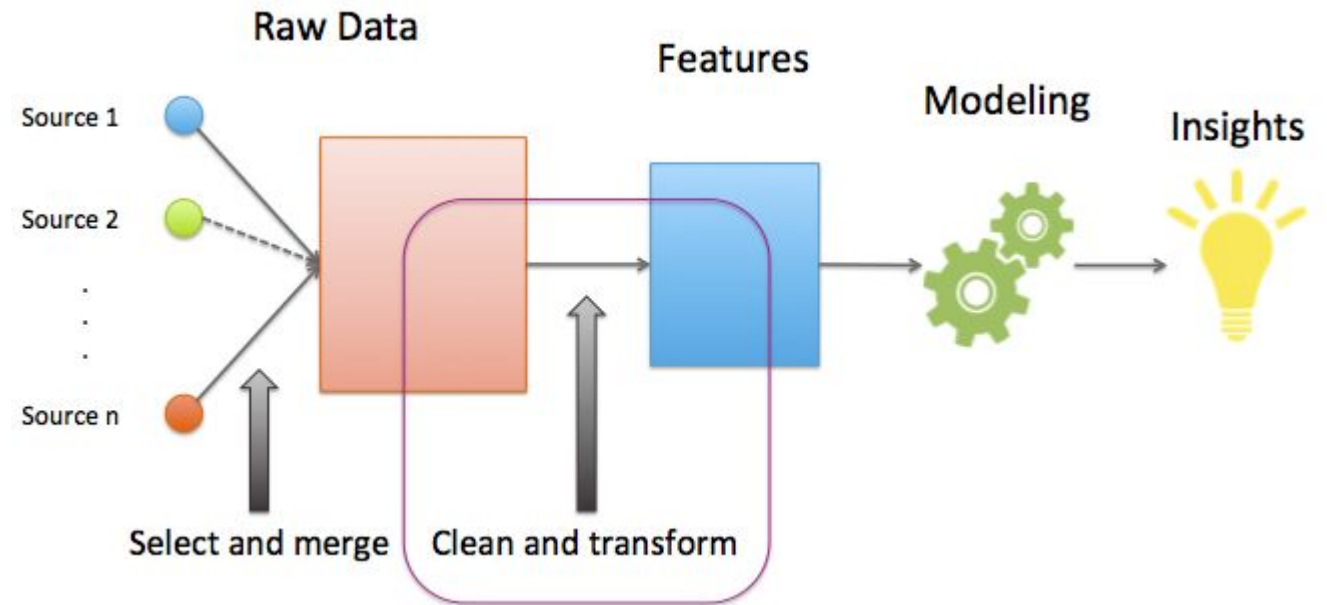
> Goal

- Explore implementations of machine learning in optical fiber fusion splicing.
- Improve upon past models.
- Step towards fusion splicing automation through AI.



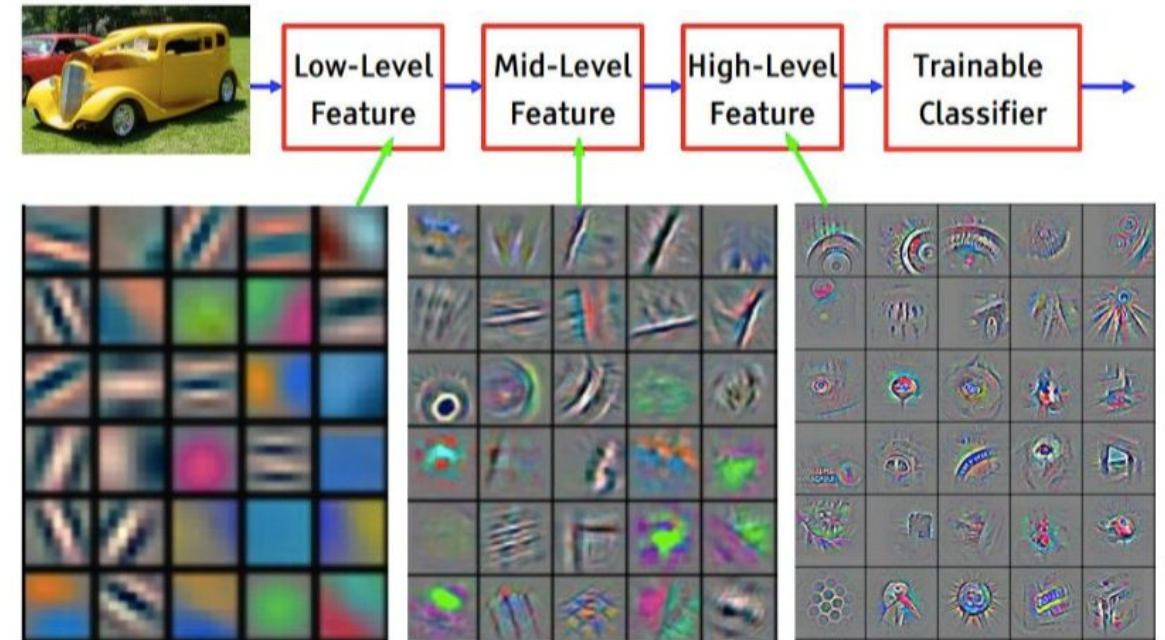
> How it works pt 1.

- Collected fiber image data sample gets prepared and then inputted into model.
- Data is cleaned and processed into readable format.
- Data is split into training and testing data and labels.



➤ How it works pt 2.

- Model does a lot of linear combinations and activations.
- Model makes predictions on testing data based on the highest probability obtained after training data.
- Comparing the number of correct predictions to incorrect gives the test accuracy.



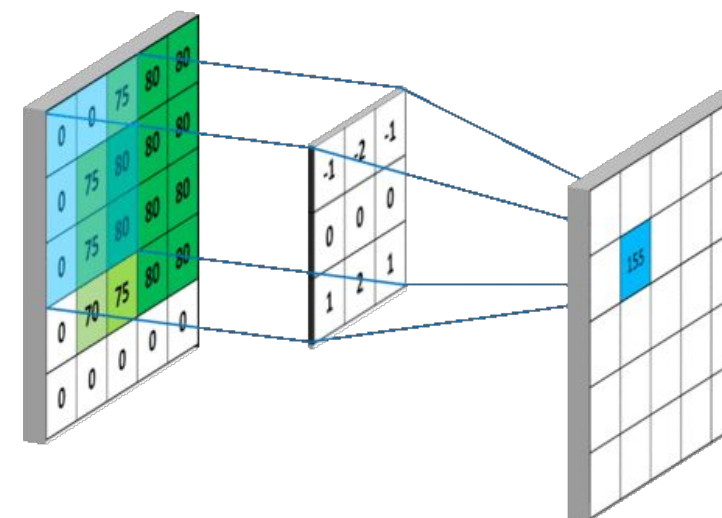
Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

> Why CNN?

Image classification task

- Neural networks work well with probabilistic tasks such as working with pixels or words.
- CNN is excellent for extracting features.
 - Ex. Ears of a dog, tail of a cat, layers of a fiber, etc.
- All information is retained
 - As opposed to an ANN which loses information in flatten layer.
- Efficient and allows for huge amounts of inputs (Pixels)
 - MLPs are fully connected and will have too many weights and connections when dealing with lots of pixels.

CNNs are perfectly designed for dealing with 2-dimensional grid structured data



Difference between ANN, CNN and RNN -
GeeksforGeeks

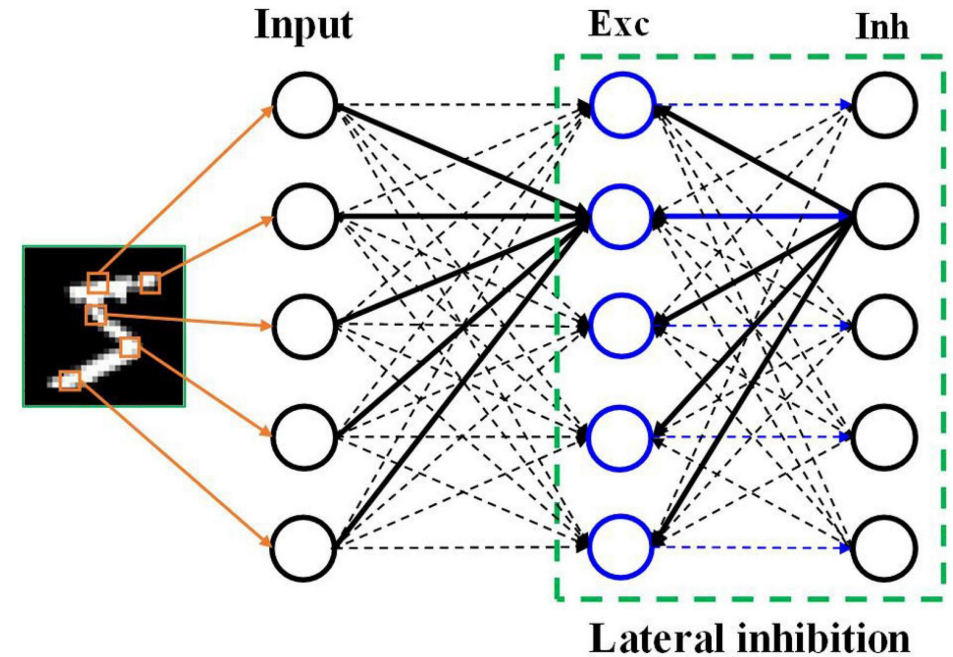
> Neural Network Takeaways

Calculations are not hard... There are just too many of them.

- Some simple models will have billions of linear combinations. (Too many for humans to do).
- There are pros and cons for every step. (Ex. Too many epochs (iterations) will lead to overfitting of training set).
- The quality of collected and prepared data makes the model.

A simple CNN for classifying a 5x5 pixel image done by hand:

[Applied Multivar Analysis Hw 2.pdf](#)



3 Fiber Model Performance

Previous Model and Changes

> Previous Model

Quentin's Model in 2019

- Two-layer CNN with Max pooling and ReLU activation
 - Input: 153x21 pixel bitmaps Output: ~96% prediction accuracy
- Binary cross entropy loss with Adam optimizer
- 30 epochs of 50 batch size

Trouble with type A and type B fiber classification

> Model Changes and Improvements

Andy 2024

- Recoded model
 - Fixed deprecated functions
- Changed file format
- Processed data to be stored efficiently
- Fixed loss function to sparse categorical cross entropy
- Retuned hyperparameters
 - Adjusted batch size to 30 and epochs to 20 for overfitting

Overall accuracy improved ~1%

Significant increase to type A, and type B prediction accuracy

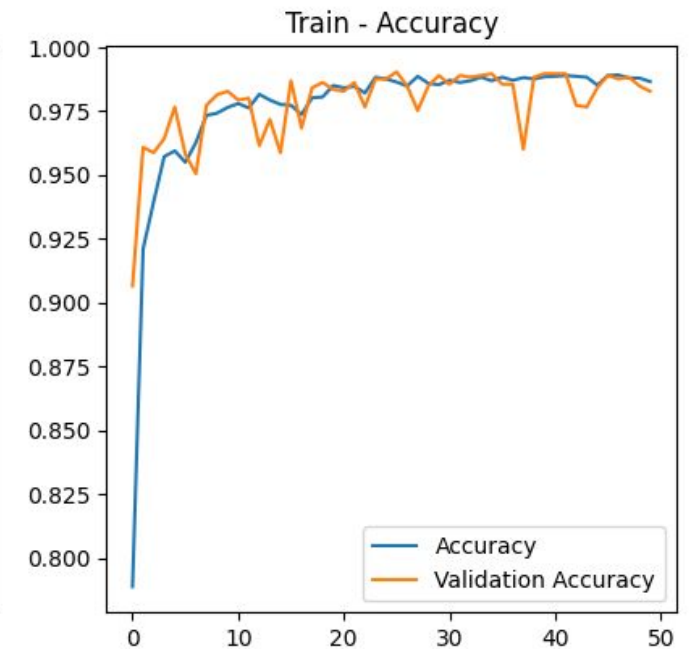
> Model performance

- We observe a consistent ~97%+ prediction accuracy on unseen data
- Loss and accuracy charts per epoch

```
score = model.evaluate(X_test,Y_test,verbose=0)
print('Test Accuracy : {:.2f}'.format(score[1]*100) + '%')
```

✓ 20.8s

Test Accuracy : 98.28%





Prediction Class = 2
Original Class = [2.]



Prediction Class = 2
Original Class = [2.]



Prediction Class = 2
Original Class = [2.]



Prediction Class = 2
Original Class = [2.]



Prediction Class = 1
Original Class = [1.]



Prediction Class = 2
Original Class = [2.]



Prediction Class = 2
Original Class = [2.]



Prediction Class = 1
Original Class = [1.]



Prediction Class = 1
Original Class = [1.]



Prediction Class = 1
Original Class = [1.]



Prediction Class = 1
Original Class = [1.]



Prediction Class = 0
Original Class = [0.]



Prediction Class = 2
Original Class = [2.]



Prediction Class = 2
Original Class = [2.]



Prediction Class = 2
Original Class = [2.]



Prediction Class = 2
Original Class = [2.]



Prediction Class = 1
Original Class = [1.]



Prediction Class = 1
Original Class = [1.]



Prediction Class = 2
Original Class = [2.]



Prediction Class = 2
Original Class = [2.]



Adding additional fibers

Introducing type D to the model

> Data collection and processing

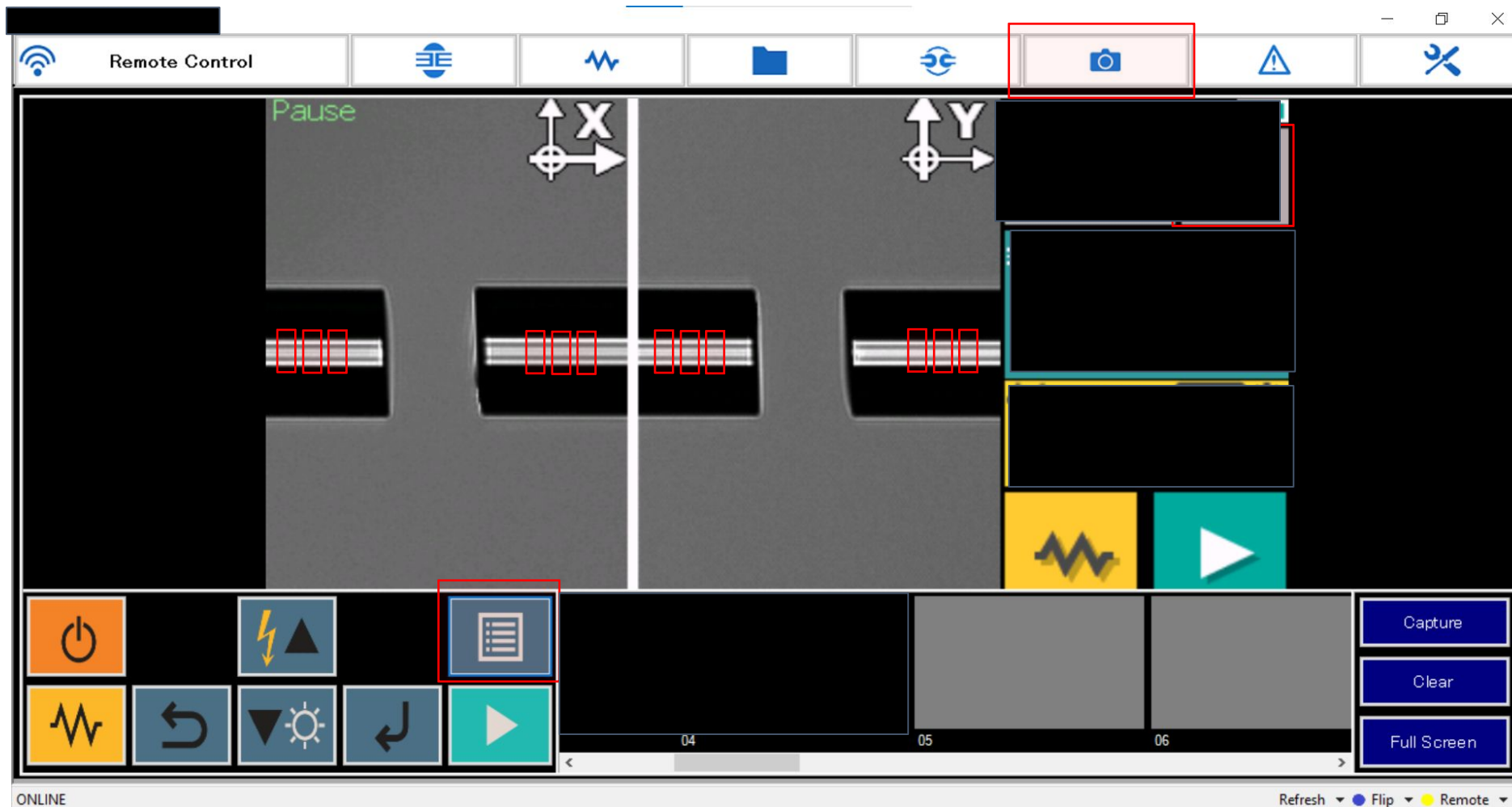
We are using the X splicer for type C data collection.

- Fiber stripping, cleaning, cleaving and capture.
- Recording unique fiber id.

Fiber_capture_conversion code to convert the image to cross-section bitmaps.

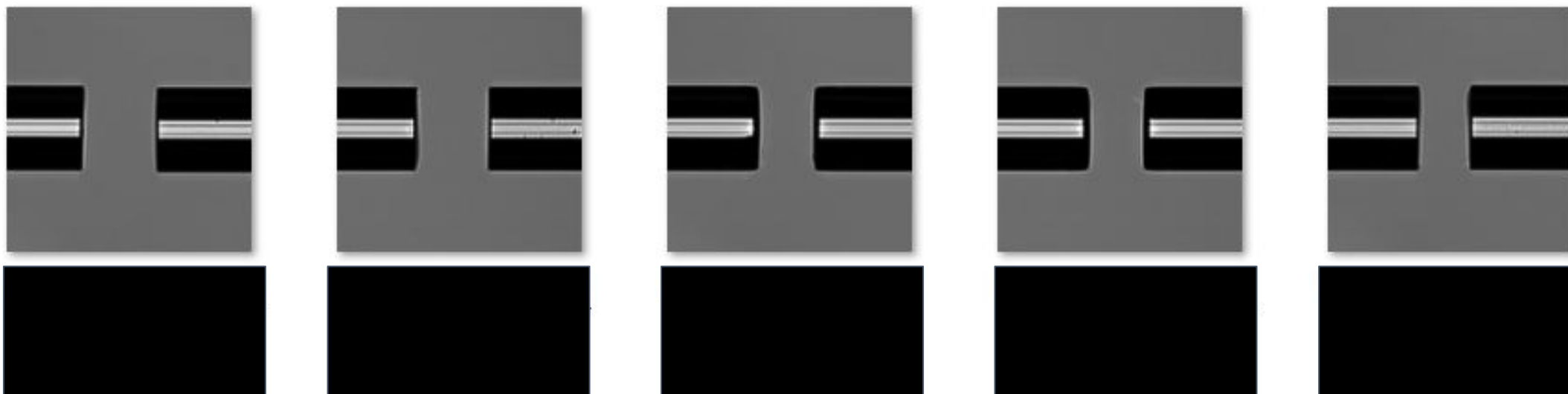
- Select 151x21 pixel cross-sections on X and Y captures.
- Convert selected cross-sections to numpy array/bitmap
- Random zoom from choice of 1x, 1.5x and 2x.
- Save and name cross-section bitmaps
[cross-section position]_[fiber id]_[X/Y].txt

> Data Collection



> Data Collection cont.

Save to folder name: [fiber type]_CAPTURES
Ex. Type C _CAPTURES



> Data Processing

Goal: Convert 512x512 png files into 151x21 bitmaps

- Take 6 cross sections (3 left , 3 right) per capture
- Name the bitmap txt files

[L1/L2/L3/R1/R2/R3]_[Fiber ID]_[X/Y].txt



```
# Change based on fiber image path
fiber_name = 'EZ'
newpath = 'C:/Users/landy/Documents/VSCODE/' + fiber_name + '_CONVERT'
if not os.path.exists(newpath):
    os.makedirs(newpath)

img_folder = 'C:/Users/landy/Documents/VSCODE/' + fiber_name + '_CAPTURES'
out_folder = newpath

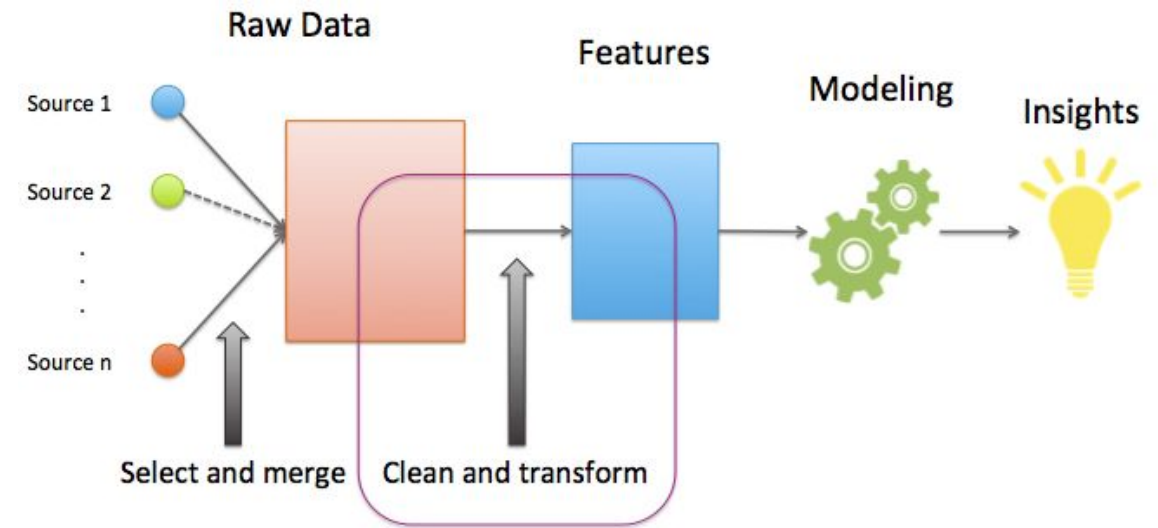
# Getting unique image name
for fiber in os.listdir(img_folder):
    fiber_path = os.path.join(img_folder, fiber)
    img = Image.open(fiber_path)
    img_arr = asarray(img)
    name = fiber[0:-4] + '.txt'

    # Get 3 different 21 pixel cross sections from left fiber end
    # Starting at 50, 100, 150 pixels
    for l in range(1, 4):
        locals()["img_arrL" + str(l)] = img_arr[180:331, 1 * 50 : 1 * 50 + 21]
        with open(os.path.join(out_folder, 'L' + str(l) + '_' + name), 'w') as file:
            for line in locals()["img_arrL" + str(l)].astype(str):
                file.write(",".join(line) + "\n")

    # Get 3 different 21 pixel cross sections from right fiber end
    # Starting at 512 - 150 - 21, 512 - 100 - 21, 512 - 50 - 21 pixels
    for r in range(1, 4):
        locals()["img_arrR" + str(r)] = img_arr[180:331, 512 - r * 50 - 21: 512 - r * 50]
        with open(os.path.join(out_folder, 'R' + str(r) + '_' + name), 'w') as file:
            for line in locals()["img_arrR" + str(r)].astype(str):
                file.write(",".join(line) + "\n")
```

> Data Processing Cont.

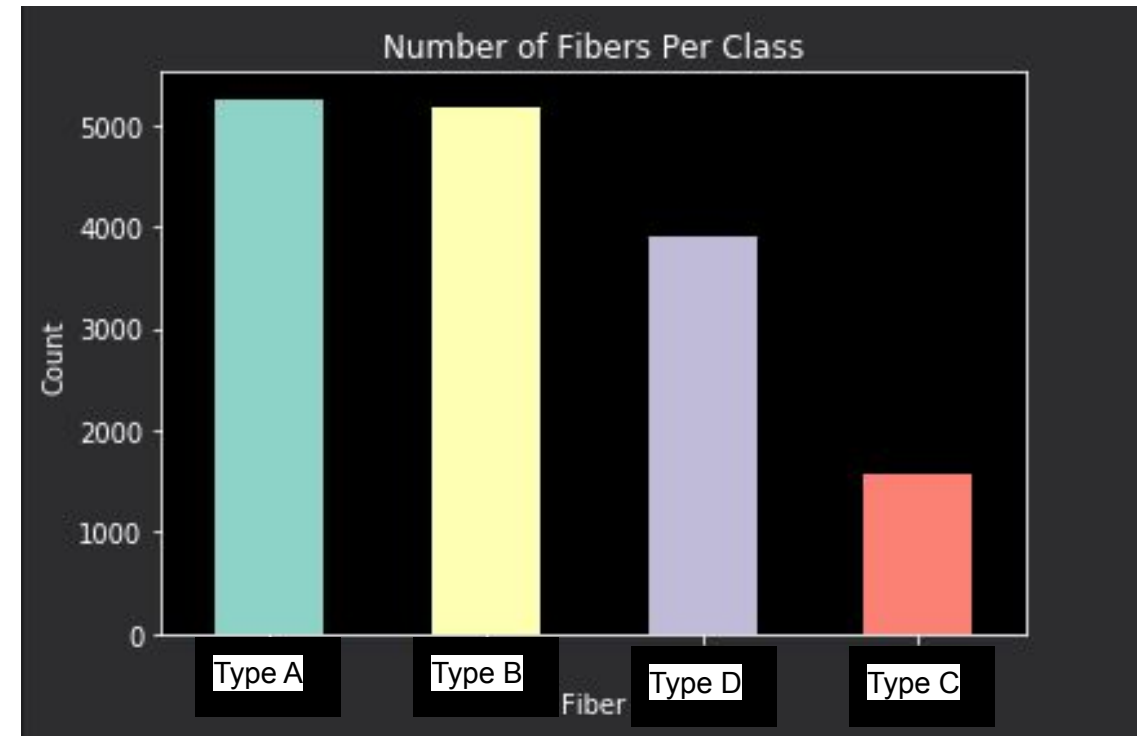
- The raw image data is now in a more compact form.
 - Easier for the machine to read.
- Bitmap txt files are fed into the model as input.



> EZ-Bend Fiber

4th Fiber addition

- Added around 1600 Type D fiber captures.
- Added imperfect splice captures for variation.
 - Model predicts dirty fibers accurately
- Accounted for class imbalance by adjusting class weights. (0.5, 0.5, 0.8, 1.0)
 - Model performs even with imbalance.



Decision tree

Classification for 4 fibers

> Decision tree classifier

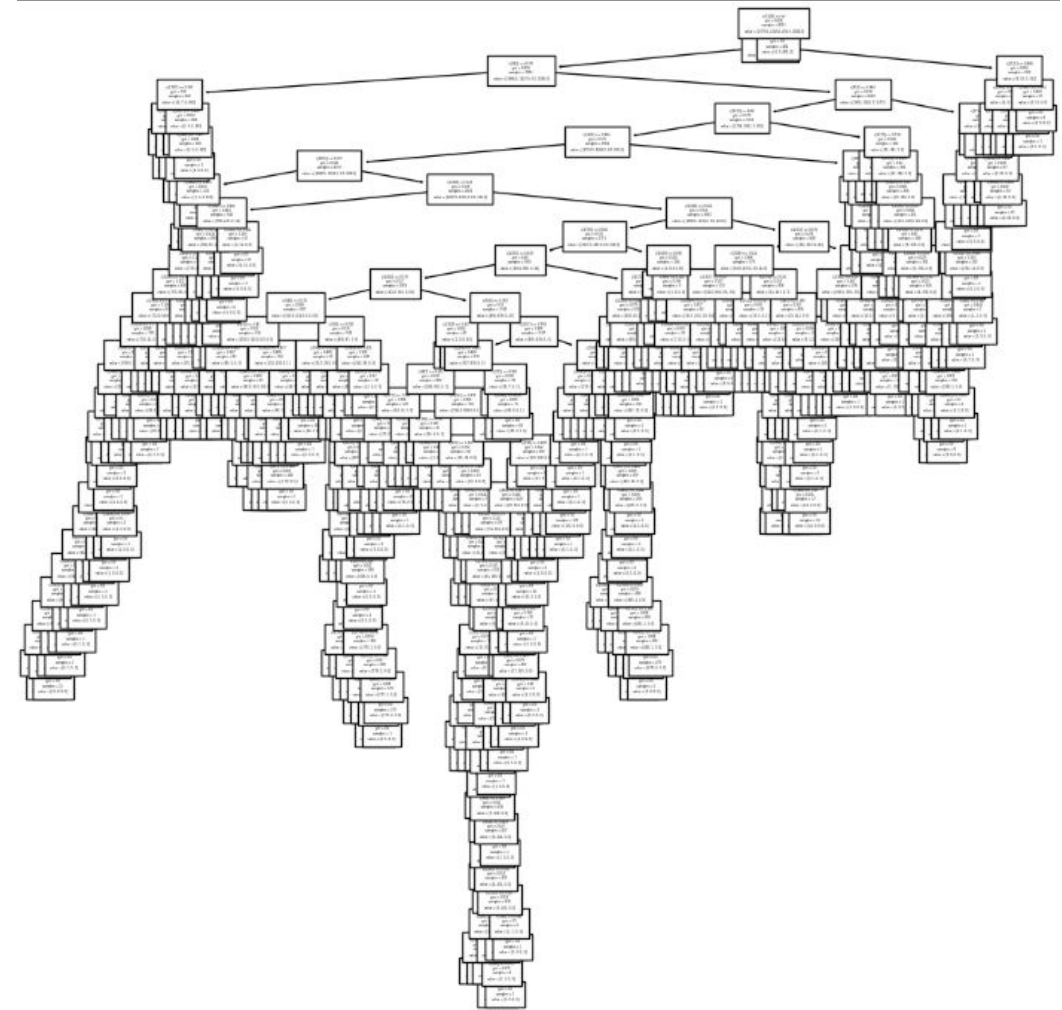
Classifies fibers by minimizing Gini impurity index

- Selects classification using leaf nodes with lowest Gini indexes.

$$I_G = 1 - \sum_{j=1}^c p_j^2$$

P_j = probability of outcome j
(classifying as fiber j)

Decision Tree in Machine Learning - GeeksforGeeks



➤ Model Performance (Decision tree)

Using a decision tree classifier, we observe only a ~90% prediction accuracy on unseen data.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
#class_weight = {0: 0.3, 1: 0.4, 2: 1.0, 3: 0.3}

# Decision tree using gini impurity index
modelTree = DecisionTreeClassifier(criterion = 'gini', splitter = 'best')

modelTree.fit(X_train, Y_train)

y_pred = modelTree.predict(X_test)
✓ 15.5s

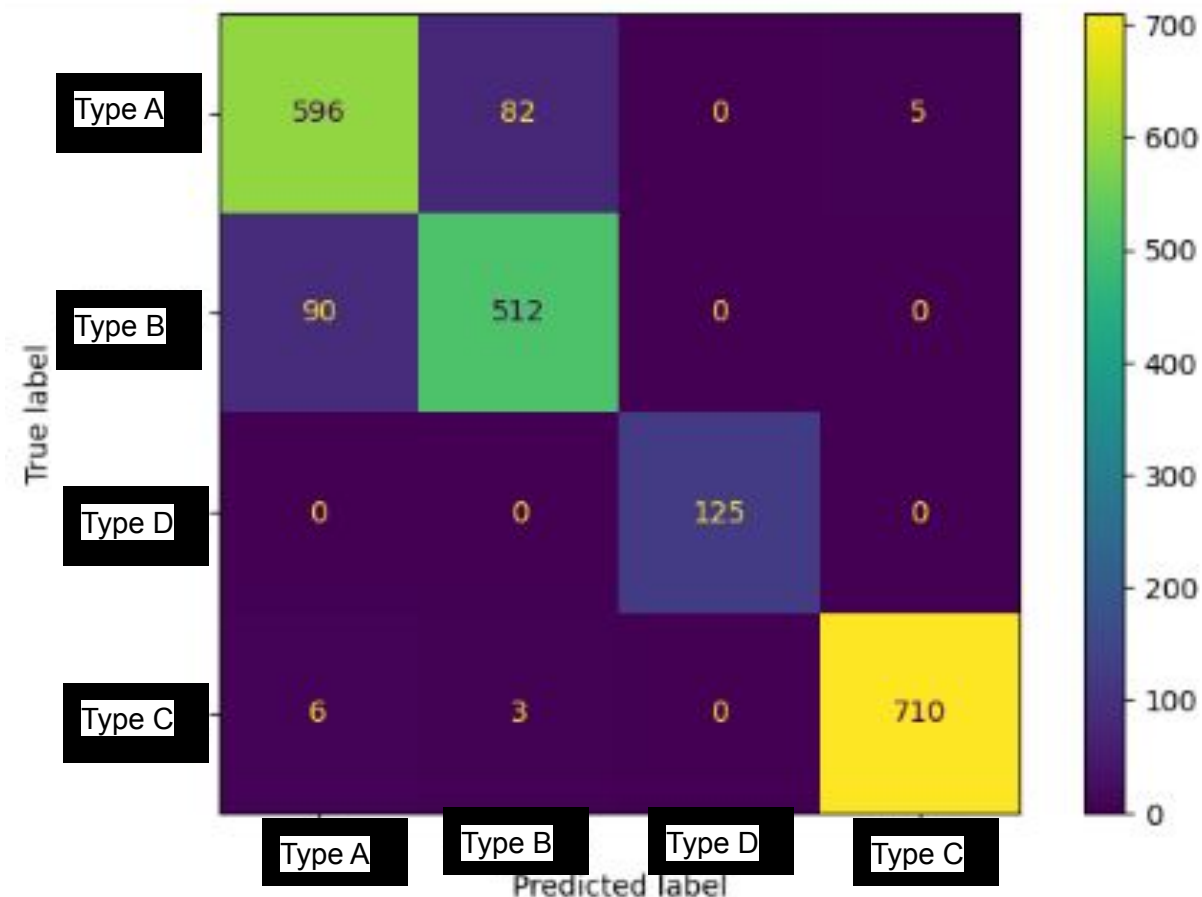
# Decision tree metrics
print("Accuracy score: ", accuracy_score(Y_test,y_pred))
print("Precision Score : ",precision_score(Y_test, y_pred,average='micro'))
print("Recall Score : ", recall_score(Y_test, y_pred, average='micro'))
✓ 0.0s

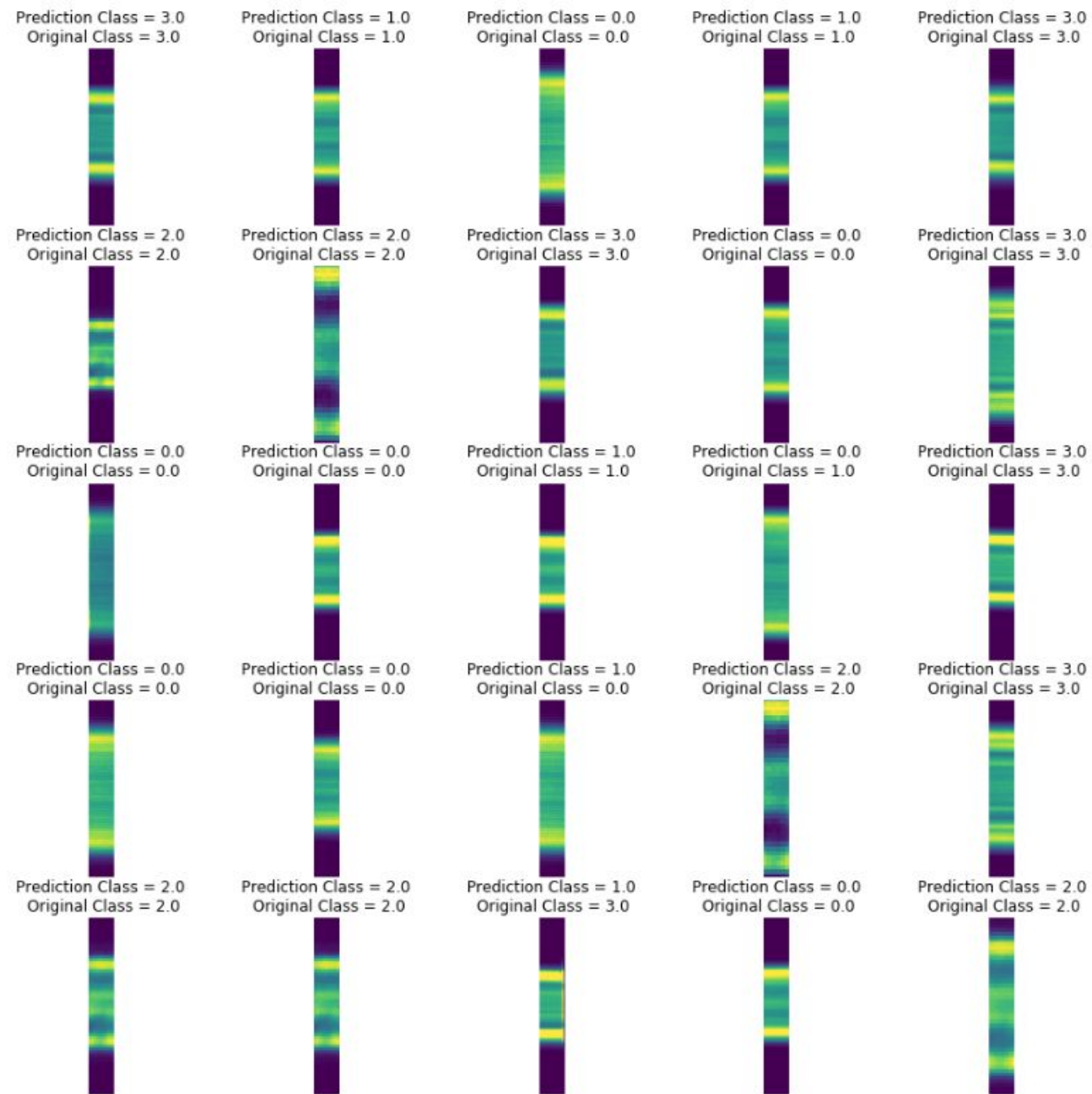
Accuracy score: 0.9126350399248473
Precision Score : 0.9126350399248473
Recall Score : 0.9126350399248473
```

> Accuracy explained (Decision tree)

Confusion matrix

- We observe accurate predictions with Type D and Type C fibers.
 - Correct predictions on the diagonal squares.
- Type A and Type B often predicted incorrectly.
 - Potential cause due to very similar fiber structures.
 - Model not complex enough.





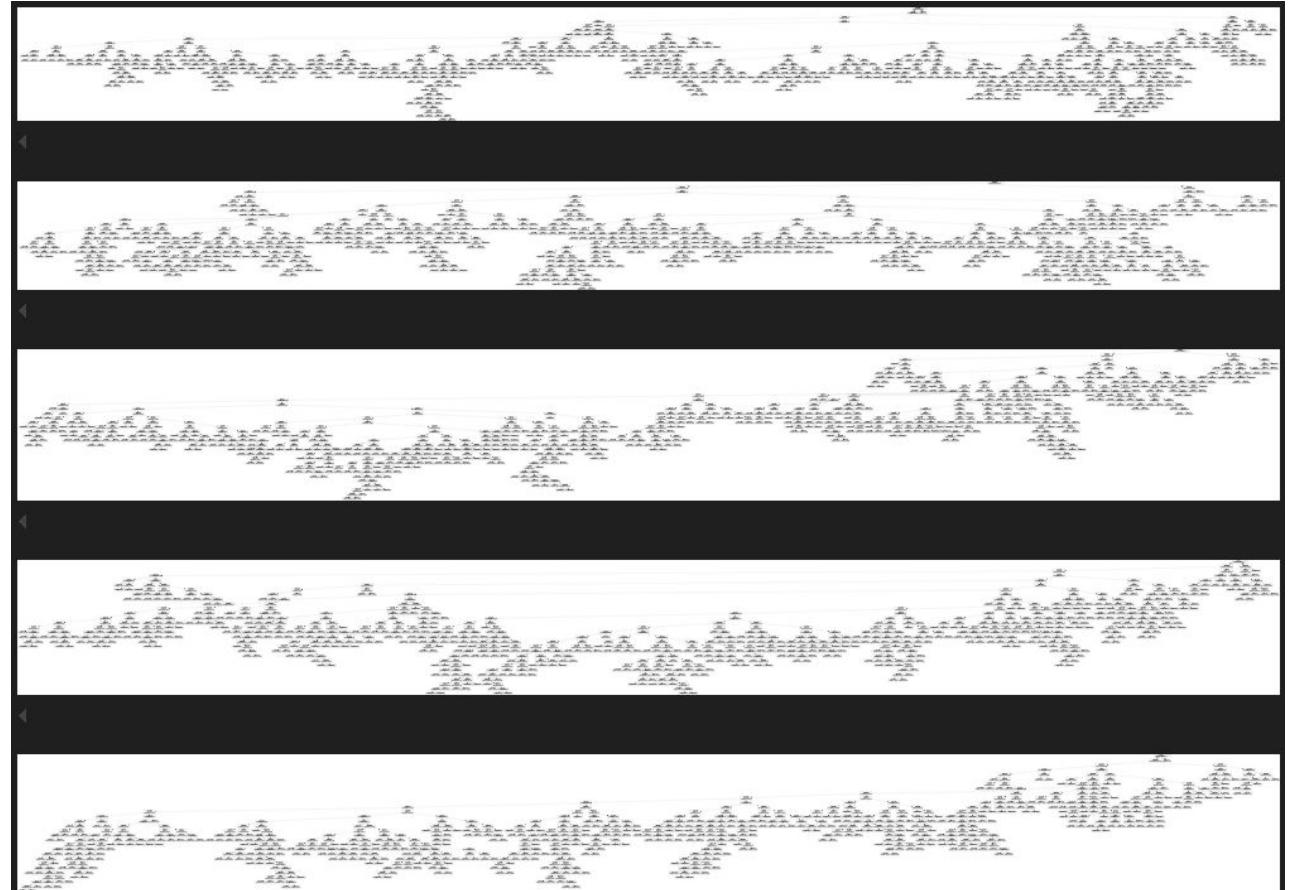
Random Forest

Classification of 4 fibers

> Random Forest Classifier

Utilizes an “army of decision trees” and random feature selection.

- Uses Gini Impurity Index per tree
- Classifies based on most frequent prediction among all trees.
- Much more complex than a single decision tree.



Random Forest Algorithm in Machine Learning - GeeksforGeeks

➤ Model Performance (Random forest)

Using a Random Forest classifier, we observe a ~97+% prediction accuracy on unseen data.

```
from sklearn.ensemble import RandomForestClassifier

randomTree = RandomForestClassifier(criterion='gini', bootstrap=True)

randomTree.fit(X_train, Y_train)

rf_pred = randomTree.predict(X_test)

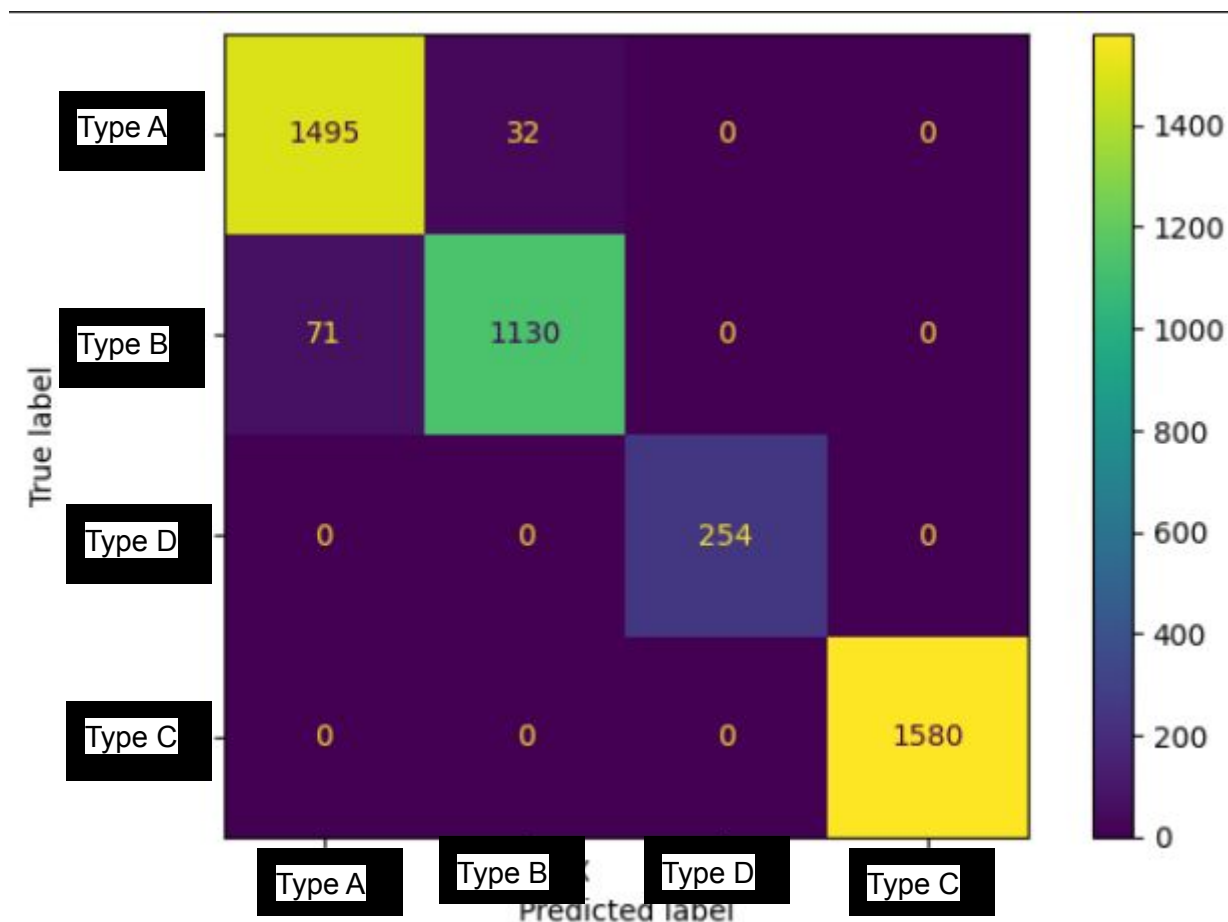
print("Accuracy score: ", accuracy_score(Y_test, rf_pred))
print("Precision Score : ", precision_score(Y_test, rf_pred, average='micro'))
print("Recall Score : ", recall_score(Y_test, rf_pred, average='micro'))
```

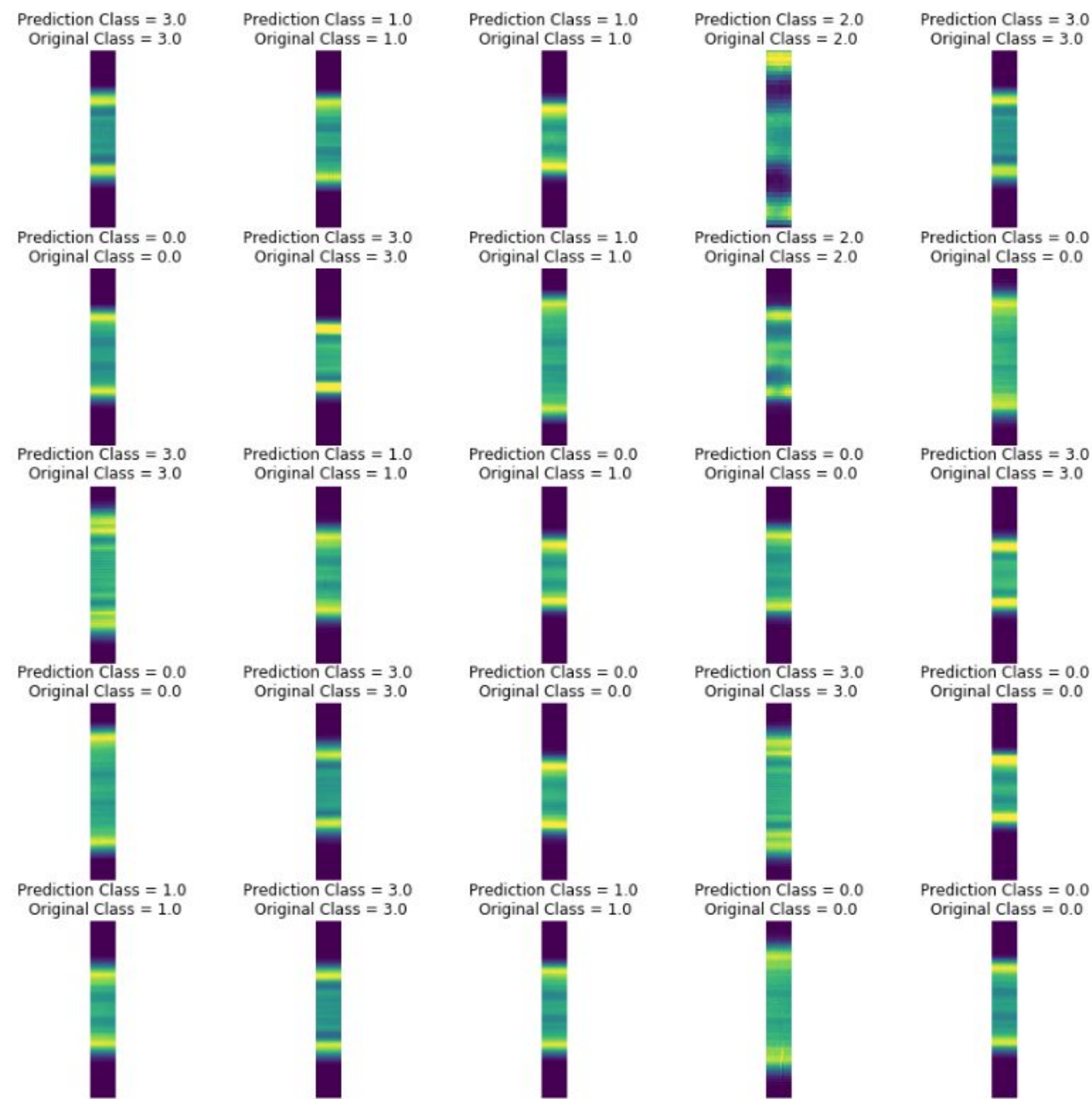
```
Accuracy score: 0.9774221832529593
Precision Score : 0.9774221832529593
Recall Score : 0.9774221832529593
```


> Accuracy explained (Random forest)

Confusion matrix

- We observe overall accurate predictions.
 - Correct predictions on the diagonal squares.
- Much better prediction accuracy than a single decision tree.
 - Model complex enough to distinguish majority of fibers.
 - Still some misclassifications of Type A and Type B.



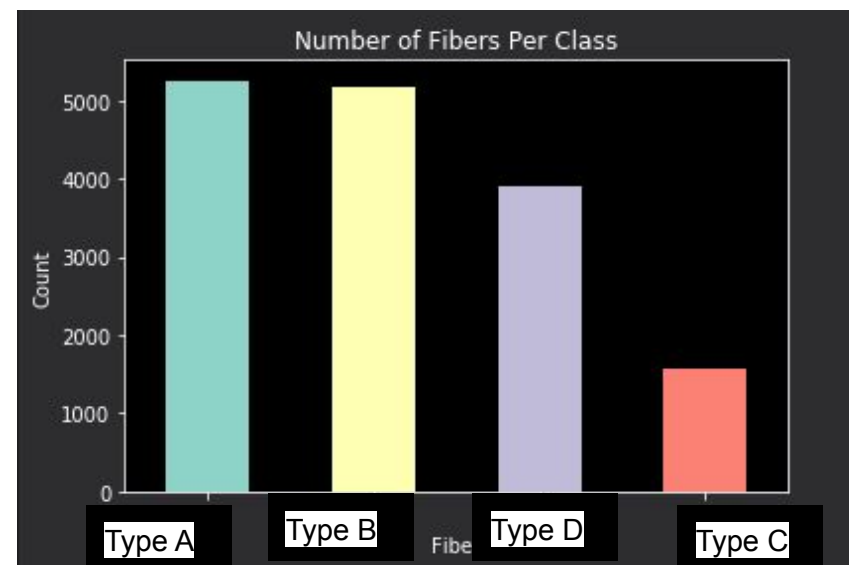
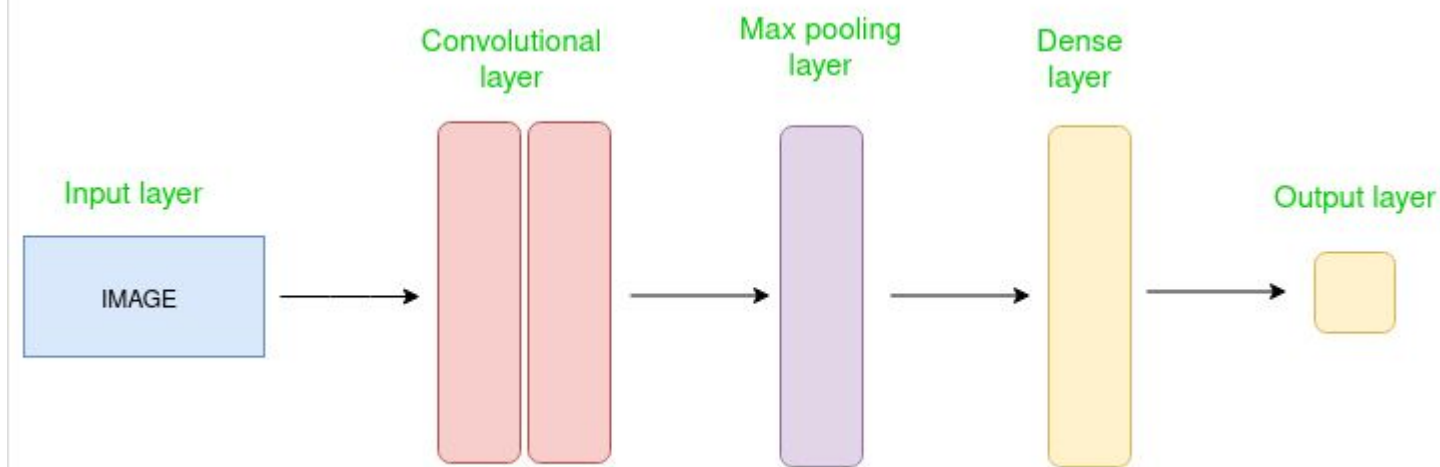


Convolutional Neural Network

Classification of 4 fibers

> CNN Classification (4 Fibers)

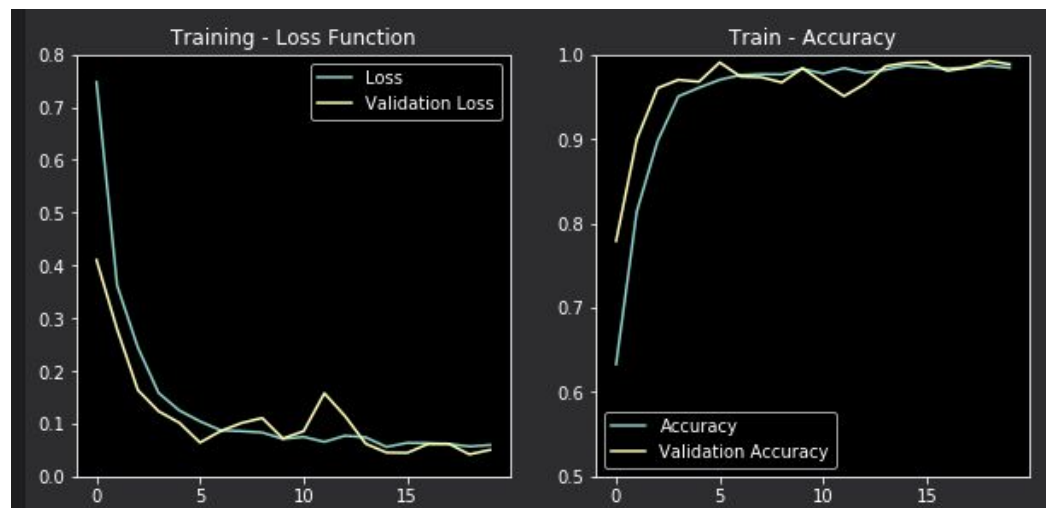
- Model structure similar to 3 fiber classification CNN.
- Retuned hyperparameters to fit new model.
- 4 Output nodes for 4 fiber classes
- Type D fiber introduced to the model.



> CNN Model Performance

We observe a ~99% prediction accuracy on unseen data

- Overall best performing model out of the three.
- Probabilistic (NN) vs deterministic (Tree) view.
- Neural networks memorize more complex probability.



```
[47]: # Training the Model...
0.0861 - val_acc: 0.9740
Epoch 8/20
8916/8916 [=====] - 23s 3ms/step - loss: 0.0857 - acc: 0.9767 - val_loss:
0.1012 - val_acc: 0.9731
Epoch 9/20
8916/8916 [=====] - 23s 3ms/step - loss: 0.0832 - acc: 0.9763 - val_loss:
0.1107 - val_acc: 0.9668
Epoch 10/20
8916/8916 [=====] - 23s 3ms/step - loss: 0.0711 - acc: 0.9830 - val_loss:
0.0717 - val_acc: 0.9839
Epoch 11/20
8916/8916 [=====] - 23s 3ms/step - loss: 0.0749 - acc: 0.9773 - val_loss:
0.0857 - val_acc: 0.9664
Epoch 12/20
8916/8916 [=====] - 23s 3ms/step - loss: 0.0654 - acc: 0.9838 - val_loss:
0.1577 - val_acc: 0.9507
Epoch 13/20
8916/8916 [=====] - 23s 3ms/step - loss: 0.0771 - acc: 0.9781 - val_loss:
0.1142 - val_acc: 0.9650
Epoch 14/20
8916/8916 [=====] - 23s 3ms/step - loss: 0.0739 - acc: 0.9818 - val_loss:
0.0618 - val_acc: 0.9861
Epoch 15/20
8916/8916 [=====] - 23s 3ms/step - loss: 0.0559 - acc: 0.9869 - val_loss:
0.0452 - val_acc: 0.9901
Epoch 16/20
8916/8916 [=====] - 23s 3ms/step - loss: 0.0635 - acc: 0.9845 - val_loss:
0.0446 - val_acc: 0.9910
Epoch 17/20
8916/8916 [=====] - 23s 3ms/step - loss: 0.0631 - acc: 0.9834 - val_loss:
0.0614 - val_acc: 0.9807
Epoch 18/20
8916/8916 [=====] - 23s 3ms/step - loss: 0.0616 - acc: 0.9846 - val_loss:
0.0619 - val_acc: 0.9848
Epoch 19/20
8916/8916 [=====] - 23s 3ms/step - loss: 0.0567 - acc: 0.9869 - val_loss:
0.0417 - val_acc: 0.9924
Epoch 20/20
8916/8916 [=====] - 23s 3ms/step - loss: 0.0595 - acc: 0.9841 - val_loss:
0.0502 - val_acc: 0.9883
```

```
# Test Accuracy
score = model.evaluate(X_test,Y_test,verbose=0)
print('Test Accuracy : {:.2f}'.format(score[1]*100) + '%')

Test Accuracy : 98.83%
```

(Neural networks work better for probabilistic calculation like pixel values or text data)

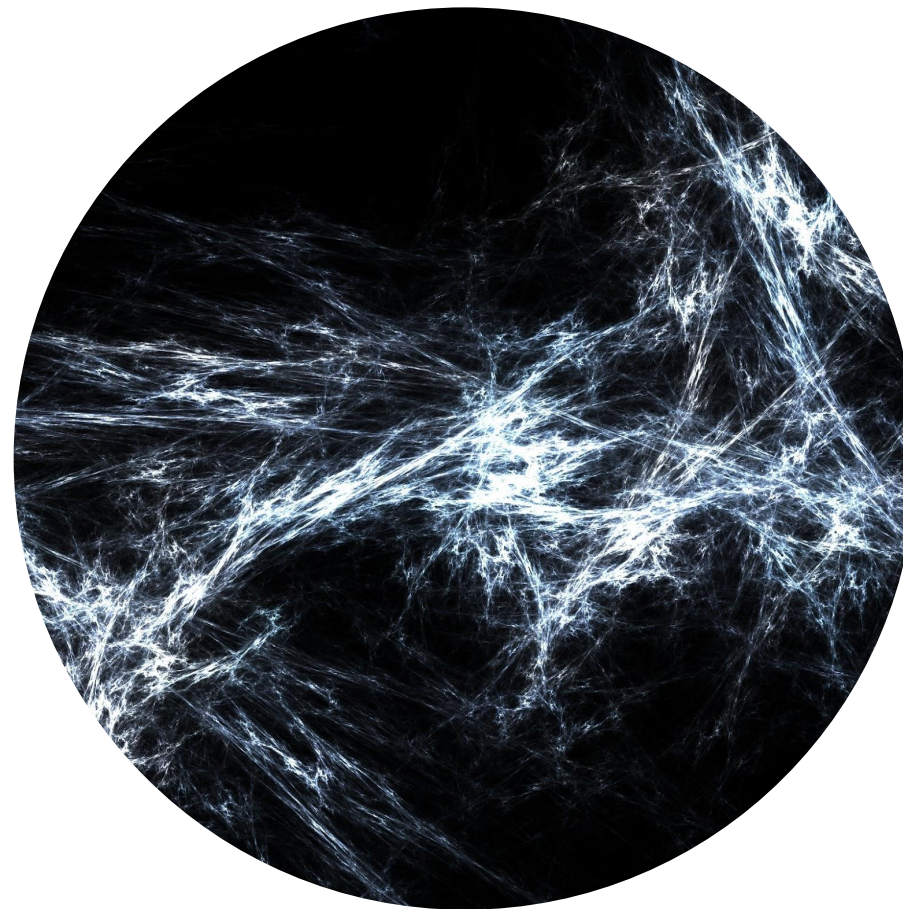


> Final takeaways

- Possible use of AI for fiber classification
- Possible classification of more fibers
- Possible classification of dirty/faulty captures
- Neural networks vs Decision tree

Future Models:

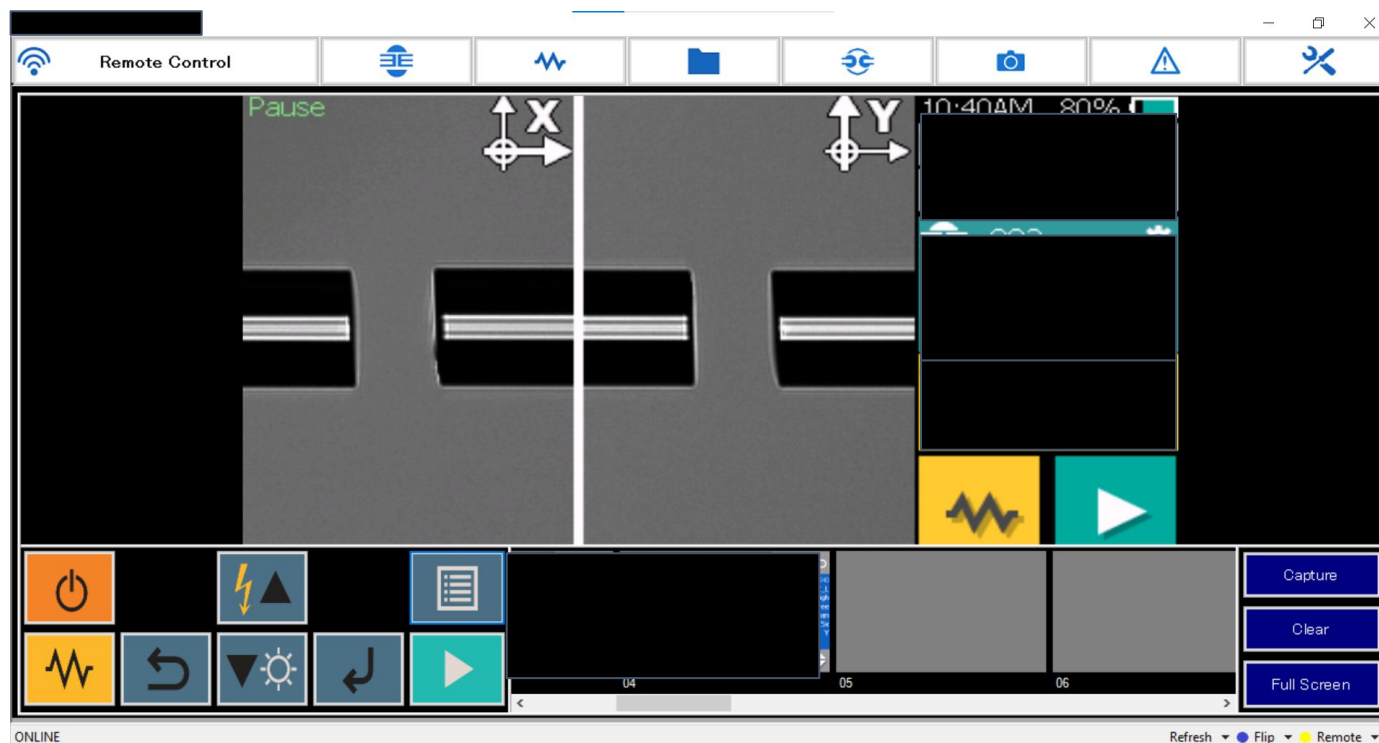
- Multilabel classification of entire splicer capture
 - Fiber recipe selection automation
- Reinforcement learning agent for splicing loss minimization.
 - Fiber position automation using AI.



> Future Models

Multilabel Fiber Classification

- Expand the fiber classification to include both sides of the capture image.
- Each multi-label class can be mapped to a fiber recipe. (Ex. AW to AW_FLEX recipe)
- Requires a CNN trained on full image captures instead of individual fiber cross-sections.



> Future Models

Splicing parameter automation

- Interacts with the splicer machine software to adjust fiber position, rotation, etc.
- Requires a reinforcement learning agent.
- The splice loss value can be used as a reward function for the agent.
- Goal is to minimize the loss value.

$$\text{Splice Loss} = \frac{\text{Splice loss}_{A \text{ to } B} + \text{Splice loss}_{B \text{ to } A}}{2} \quad \text{Eq. (1)}$$

Example:

$$\text{Splice Loss} = \frac{0.04\text{dB}_{A \text{ to } B} + (-0.02\text{dB})_{B \text{ to } A}}{2} = 0.01 \text{ dB} \quad \text{Eq. (2)}$$



Thank You

Any Questions?



www.ofsoptics.com

OFS PROPRIETARY AND CONFIDENTIAL

Copyright © 2023 OFS Fitel, LLC. All Rights Reserved.